

Chap: URL Routing And Dependency Injection

Putting URL Routing in context:

Question

Answer

- a) What is it? - URL routing consolidates the processing and matching of URLs, allowing components known as endpoints to generate responses.
- b) Why is it useful? - URL routing obviates the need for each middleware components to process the URL to see whether the request will be handled or passed along the pipeline. The result is more an efficient and easier to maintain.
- c) How is it used? - The URL routing middleware components are added to the request pipeline and configured with a set of routes. Each route contains a URL path and a delegate that will generate a response when a request with the matching path is received.
- d) Are there any pitfalls or limitations? - It can be difficult to define the set of routes matching all the URLs supported by a complex application.
- e) Are there any alternatives? - URL routing is an optional, and custom middleware components can be used instead.

Adding the Routing Middleware and defining an Endpoint:

- The routing middleware is added using two separate methods: UseRouting and Endpoints.

- The UseRouting method adds the middleware responsible for processing requests to the pipeline.
- The Endpoints method is used to define the routes that match URLs to endpoints.
- Any modification to the incoming request and data, that routing depends on, can be modified only before UseRouting.
- UseRouting will use the routing data to select appropriate, best-matching endpoint.
- Any middleware, which comes after UseRouting and before UseEndpoints, can use endpoint information to apply some additional checks before the endpoint is executed. This might include checking CORS configurations, authentication and authorization checks, etc.
- UseEndpoints is a terminal middleware, provided the endpoint is existing in the application. This means any middleware after it would not be executed if the endpoint was found.
- Any middleware after UseEndpoints middleware would be executed only for those requests for which endpoints were not configured.

The IEndpointRouteBuilder Extension Methods:

Name	Description
(1) MapGet(pattern, endpoint)	- This method routes HTTP GET requests that match the URL pattern to the endpoint.
(2) MapPost(pattern, endpoint)	- This method routes HTTP POST requests that match the URL pattern to the endpoint.

(6) Map (pattern, endpoint) - This method routes all HTTP requests that match the URL pattern to the endpoint.

`http://localhost:5000/mauting` ← path

http://localhost:5000/capital/UK ← Second Segment
↑
First Segment

app.MapGet("/{Capital}/{UK}", new...

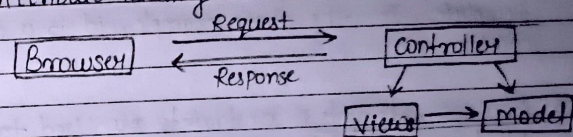
URL Path	Description
----------	-------------

- (a) /capital - No Match - too few segments.
(b) /capital/europe/uk - No Match - too many segments.

(c) /name/uk - No match - first segment is not capital.
(d) /capital/uk - Matches.

• Routing Techniques:

- conventional Routing
- Attribute Routing



http://localhost:1234/Home/Index

```
public class HomeController : Controller
{
    // GET: /Home/
    public ActionResult Index()
    {
        return View();
    }
}
```

```
private IEmployeeRepository _employeeRepository;  
public HomeController (IEmployeeRepository  
employeeRepository) ...
```

```
public IActionResult Index() {  
    var model = -EmployeeRepository.  
        GetAllEmployees();  
    return View(model);  
}
```

3

Key Components of Routing in ASP.NET Core:

- (1) Route Template: A Route Template defines the structure of a URL and includes placeholders for route values. For example, `/Products/{id}` is a route template with a placeholder `{id}`.
- (2) Route Values: Route values are extracted from the URL based on the placeholders defined in the route template. In the above example, `id` is a route value.
- (3) Route Constraints: Constraints are used to restrict which URLs match a route template. For instance, you can constrain `id` to only match numeric values.
- (4) Route Parameters: Route parameters are placeholders in controller action methods that correspond to route values. They receive values from the route values extracted from the URL.

Example: Creating Routes and Controller

- Let's create a simple ASP.NET Core application to demonstrate routing with controllers and actions.

Step 1: Create a New ASP.NET Core Project.

- You can create a new ASP.NET Core project using Visual Studio or the .NET CLI.
- Commands:
`dotnet new mvc -n MyRoutingApp`
`cd MyRoutingApp`

Step 2: Create a Controller

- Add new controller named `ProductController` with an action method called `Details`.

- Example,

Using Microsoft.AspNetCore.Mvc;

```
public class ProductController : Controller {  
    public IActionResult Details(int id) {
```

```
// Simulate fetching product details from a DB.
```

```
    var product = new { Id = id, Name = "Sample  
        product" };;
```

```
    return View(product);
```

```
    }  
}
```

Step 3: Define a Route

- In the `Startup.cs` file, you can define a custom route for the `ProductController`.

- Example,

```
app.UseEndpoints(endpoints =>
```

```
{
```

```
    endpoints.MapControllerRoute(  
        name: "Product",
```

```
        pattern: "Product/{id:int}",
```

```
        defaults: new { controller = "Product",  
            action = "Details" });
```

```
    endpoints.MapControllerRoute(  
        name: "default",
```

```
        pattern: "{controller=Home}/{  
            action=Index}/{id?}");
```

```
});
```


- In this route, we specify that URLs with the pattern "product/{id:int}" should be routed to the ProductController and the Details action method, where id is constrained to be an integer.

Step 4: Create a View

- Create a View of the Details action method, in the Views/Product folder, create a file named Details.cshtml.

Example

@model dynamic

<h1> Product Details </h1>

<P> Product ID: @model.Id </P>

<P> Product Name: @model.Name </P>

Step 5: Test the Application

- Run the application and navigate to /product/123, your Web browser. You should see the product details page with the ID "123" displayed.

Middleware in .NET Core:

What is a Middleware ?

- Middleware is just a simple class that consists of an encapsulated information into an application pipeline to deal with the http request-response pipeline.
- Every part of this class has the option to pick whether to give the request to the next pipeline and, can play out specific activities when the following requests pass in the pipeline.

- In this case, request delegates are utilized to construct the request pipeline. The request delegates handle every HTTP requests.

- Request delegates are designed to configure, the Run, map, and use method on the IApplicationBuilder types that is passed, into the configure method in the Startup class. An individual request delegate can be determined in-line as a lambda expression, or it can be designed in a reusable class.

- These reusable classes are middleware or middleware components. Each middleware part in the request pipeline is responsible for invoking the next component in pipeline, or short-circuiting the pipeline.

Middleware Order of Executing :

- The middleware order of executing plays a very important role, so you have to take care of it very precisely, otherwise you may get unexpected behavior.

- So, Suppose you have 3 middleware - M1, M2 & M3 in sequential order, then it will invoke in the same order and then responds in the reverse direction.

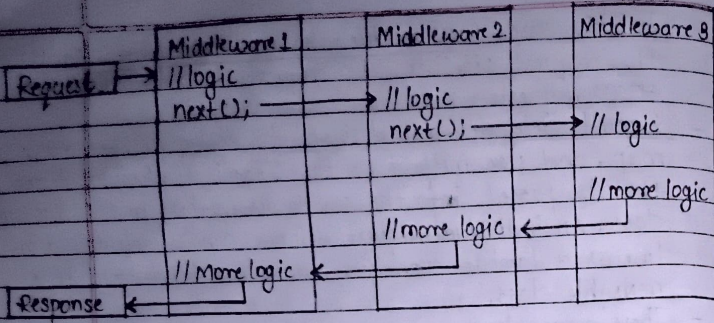
- Let's take another example,

app.UseAuthentication();

app.UseAuthorization();

- Create middleware using IApplicationBuilder, we can create middleware in the configuration method of the Startup class using the IApplicationBuilder interface.

- Here, In this following example, It will add a simple middleware using the Run method that will return a string "Hello my first method", on every requests.



for example,

```
public class Startup {
    public Startup() {}
    public void configure (IApplicationBuilder app,
        IHostingEnvironment env, ILoggerFactory loggerFactory)
    {
        // here is how to configure middleware using IApplicationBuilder
        // with run method.
    }
}
```

// I will explain run method in details just a bit.

```
app.Run(async (context) => {
    await context.Response.WriteAsync(
        "Hello my first middleware"); });
```

// I have removed here, other code for the clarity....
}

Understanding the Run, use and map method:

- (1) app.Run() Method: This middleware component may expose Run [Middleware] method that are executed at the end of the pipeline. Generally, this acts a terminal middleware is added at the end of the request pipeline, as it can't call the next middleware.
- (2) app.Use() Method: This is used to configure multiple middleware. Unlike app.Run(). We can include the next parameter into it, which calls the next request delegate in pipeline. We can also short-circuit (terminate) the pipeline by not calling the next parameter.

Example,

Let's consider the following example with the app.Use() and app.Run() and observe the output / response.

```
public void Configure (IApplicationBuilder app,
    IWebHostEnvironment env) {
    app.Use(async (context, next) => {
        await context.Response.WriteAsync("Before
        Invoke from 1st app.use()\n");
        await next();
        await context.Response.WriteAsync("After
        Invoke from 1st app.use()\n");
    });
    app.Use(async (context, next) => {
        await context.Response.WriteAsync("Before
        Invoke from 2nd app.use()\n");
    });
}
```



```
await next();
await Context.Response.WriteAsync("After
Invoke from 2nd app.Use() \n");
});
```

```
app.Run(async (context) => {
    await Context.Response.WriteAsync("Hello from
    1st app.Run() \n");
});
```

```
// the following will never be executed
app.Run(async (context) => {
    await Context.Response.WriteAsync("Hello
    from 2nd app.Run() \n");
});
```

(3) app.Map() Method: These extensions are used as a convention for branching the pipeline. The map branches the request pipeline based on matches of the given request path. If the request path starts with the given path, the branch is executed.

Example, Let's consider the following example with the app.Map() and observe the output/responses.

```
public void Configure(IApplicationBuilder app,
    IWebHostEnvironment env) {
    app.Map("/m1", HandleMapOne);
    app.Map("/m2", app.Map => {
        appMap.Run(async context => {
            await Context.Response.WriteAsync("Hello
            from 2nd app.Map()");
        });
    });
}
```

```
app.Run(async (context) => {
    await context.Response.WriteAsync("Hello
    from app.Run()");
});
```

```
}
private static void Handler ppOne (IApplication-
    Builder app) {
    app.Run(async context => {
        await context.Response.WriteAsync("Hello
        from 1st app.Map()");
    });
}
```

Exception Handling in C#:

<u>Keyword</u>	<u>Definition</u>
(a) <u>try</u>	- Used to define a try block. This block holds the code that may throw an exception.
(b) <u>catch</u>	- Used to define a catch block. This block catches the exception thrown by the try block.
(c) <u>finally</u>	- Used to define the finally block. This block holds the default code.
(d) <u>throw</u>	- Used to throw an exception manually.

1. Ex: // C# program to show how exceptions occurs in a program:

```
using System;
class GFG {
```



```

Static void main(String [] args) {
// Declare an array of max index 4
int [] arr = {1, 2, 3, 4, 5};
// Display values of array elements
for (int i = 0; i < arr.Length; i++) {
    console.WriteLine(arr[i]);
}
// Try to access invalid index of array
console.WriteLine(arr[7]);
// An exception is thrown upon executing to
// the above line
}

```

2. Ex: // Exception handling of above code using try-catch blocks:

```

using System;
class Program: System.Exception {
    static void main(String [] args) {
// Declare an array of max length index 4
int [] arr = {1, 2, 3, 4, 5};
// Display values of array elements
for (int i = 0; i < arr.Length; i++) {
    console.WriteLine(arr[i]);
}
try {
// Try to access invalid index of array
console.WriteLine(arr[7]);
// An exception is thrown upon executing to the
// above line
} catch (IndexOutOfRangeException e) {
// The message property of the object of type
// IndexOutOfRangeException is used to display the

```

```

// type of exception that has occurred to the user.
console.WriteLine("An Exception has Occurred: {0}",
    e.Message);
}
}

```

3. Ex: // C# program to show use of multiple try-catch blocks.

```

using System;
class Program {
    static void main(String [] args) {
        int [] arr = {1, 0, 75, 52};
        try {
// Try to generate an exception
for (int i = 0; i < arr.Length; i++) {
    console.WriteLine(arr[i] / arr[i+1]);
}
} catch (IndexOutOfRangeException e) {
    console.WriteLine("An Exception has Occurred:
        {0}", e.Message);
}
// catch block for atleast attempt to divided by zero
catch (DivideByZeroException e) {
    console.WriteLine("An Exception has Occurred: {0}",
        e.Message);
}
// catch block for value being out of range
catch (ArgumentOutOfRangeException e) {
    console.WriteLine("An Exception has Occurred:
        {0}", e.Message);
}
}

```


4. ex: // User Defined Exception

// C# program to show the user defined exceptions.

using System;

// user defined Exception class

// child of Exception

class DivideByZero : Exception {

// constructor

public DivideByZero() {

console.WriteLine("Exception has occurred:");

}

}

class Program {

// Method to perform Division

public double DivisionOperation(double numerator,
double denominator) {

// throw exception when denominator value is 0.

if (denominator == 0) {

throw new DivideByZero(); }

// Otherwise return the result of the division

return numerator/denominator;

}

// Main

static void Main(String[] args) {

Program obj = new Program();

double num = 9, den = 0, quotient;

try {

// code block that may cause an exception

quotient = obj.DivisionOperation(num, den);

console.WriteLine("Quotient = {0}", quotient);

}

// catch blocks to catch the generic exception

catch (Exception e) {

// Message property of exception object e will

// give the specific type of the exception

console.WriteLine(e.Message);

}

}

}

chap: Security and Deployment in ASP.NET Core

Creating Identity, Authorization Policies, and Deployment Steps

Creating the Identity Database:

- ASP.NET Core Identity is used for authorization and authentication.
- Steps:
 - Add Identity packages:
Microsoft.AspNetCore.Identity, EntityFrameworkCore
 - Configure Services in Program.cs or Startup.cs
 - Apply Migration: ef migrations add, ef database update.

Adding a Conventional Administration Feature

- Create 'Admin' role and assign it to a user.
- Code Snippets:
 - roleManager.CreateAsync(new IdentityRole("Admin"))
 - userManager.AddToRoleAsync(user, "Admin")

• Use `[Authorize(Roles = "Admin")]` on admin Only pages.

Applying a Basic Authorization Policy:

- Define policies in `Program.cs`:
 - Options. `AddPolicy("OnlyAdmins", Policy => Policy.RequireRole("Admin"))`.
- Apply With `[Authorize(policy = "OnlyAdmins")]` on controllers/action.

Creating the Account Controller & Views:

- Use Scaffolding: `aspnet-codegenerator identity -dc ApplicationDbContext`
- Or manually create Account Controller With login/logout functionality.
- Use `SignInManager` and `UserManager` for user operations.

Testing the Security Policy:

- Check Unauthorized user redirection to login page.
- Validate access denied for non-admins.
- Test login/logout and role-based navigation manually or via unit tests.

Preparing ASP.NET Core for Deployment:

- Set environment:
`ASPNETCORE-Environment = production`
- Update appsettings. `production.json`
- Publish: `dotnet publish -c Release -o ./Publish`

- Host Using `ITS, Azure, Docker, or Other Services`.
- Configures `HTTPS, logging, and error handling`.

Summary table:

- Identity DB: Add Packages, configure, migrate.
- Admin features: Create Role, assign user, use `[Authorize]`.
- Policy: Custom rules, role-based access.
- Account: Login/logout using Identity.
- Testing: Redirects, access control Verification.
- Deployment: Environment - set up, publish, hosting.

Unit-4

CHAPTER: ASP.NET Core: RESTful Web Services

- Detailed concepts and features Overview

Introduction to RESTful Web Services:

- REST: Representational State Transfer.
- Stateless architecture using HTTP methods.
- Used for a building Scalable web Service.

Key Principles of REST:

- (1) Stateless
- (2) Client-Server Architecture
- (3) Cacheable
- (4) Uniform Interface
- (5) Layered Systems

HTTP methods in REST:

- (1) GET - Retrieve data.
- (2) POST - Create a New resources.
- (3) PUT - Update a resources completely.
- (4) PATCH - Partially Updates a resources.
- (5) DELETE - Remove a resources.

4 SQL Commands (Create, update, delete, Insert).

RESTful URLs and Resources:

- Use nouns not Verbs: /products instead of /getProducts.
- Each URL points to a resources.
- Resources are represented in formats like JSON, and XML.

What is Minimal API?

- New features in ASP.NET Core 6+.
- Simplified Syntax for lightweight services.
- Defined directly in a program.cs.

Creating a Minimal API:

- `app.MapGet("/products", () => ...);`
- No need for controllers.
- Use lambda expressions for logic.

Benefits of Minimal APIs:

- Faster startup and resources time.

- Ideal for microservices and small projects.
- Less boilerplate code.

Using controllers for APIs:

- Traditional and structured approach.
- Use [ApiController] attribute.
- Better Suited for large-scale applications.

Creating a controller-based API:

- [HttpGet], [HttpPost], etc. for routing.
- Return types: IActionResult or Specific types.
- Supports filters, Validation, and model binding.

Dependency Injection in Web Services:

- Inject services into controllers or handlers.
- Improves testability and maintainability.
- Use constructor injection for services.

Asynchronous Programming in APIs:

- Use `async/await` for I/O-based (bound) operations.
- Improves scalability and responsiveness.
- Examples: `async Task <IEnumerable<Products>>`.

Validation and Error Handling:

- Use DataAnnotations for model validation.
- Check `ModelState.IsValid`.
- Handle exceptions gracefully using middleware or filters.

API Versioning:

- Supports multiple versions of APIs.
- Use URL, query string, or headers for versioning.
- Package: Microsoft.AspNetCore.Mvc.Versioning

Working With Related Data:

- Use EF Core Include() method.
- Avoid circular references during serialization.
- Use DTOs to manage related data output.

Using DTOs and AutoMapper:

- DTOs: Data Transfer Objects to shape responses.
- AutoMapper simplifies mapping between models and DTOs.
- Avoids over-posting and circular dependencies.

Supporting HTTP PATCH method:

- Allows partial updates to a resource.
- Use JSONPatchDocument and ApplyTo().
- Requires additional configuration in ASP.NET Core.

Content Negotiation:

- Automatically selects response format (JSON, XML, etc).
- Based on Accept header from client.
- Default format is JSON.

Adding XML Support:

- Add XmlSerializerFormatters() in program.cs
- Clients can request application/xml format
- Helps interoperability with older systems.

* XML file Examples:

1.

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Remainder</heading>
  <body>Don't forget me, this weekend.</body>
</note>
```
2.

```
<note>
  <to>Tove1</to>
  <from>Jani1</from>
  <heading>Remainder1</heading>
  <body>Don't forget me, this weekend1.</body>
</note>
```

Authentication in APIs:

- Use JWT (JSON Web Tokens) for ^(auth)Stateless authentication.
- Include tokens in Authorization header.
- Validation token on each request.

Authorization in APIs:

- Role-based and policy-based authorization.
- Use [Authorize(Roles = "Admin")] or custom policies.
- Combine with claims for granular access.

API Documentation With Swagger:

- Swagger (OpenAPI) provides interactive docs.
- Install Swashbuckle.AspNetCore.
- AddSwaggerGen() and UseSwagger().

Testing APIs With Swagger UI:

- Test endpoints without Postman.
- Submit requests and view responses interactively.
- Great for developers and QA (Quality Assurance).

Preparing for deployment:

- Set ASPNETCORE_ENVIRONMENT = Production.
- Use appSettings.Production.json for config.
- Run dotnet publish for release build.

Hosting web services:

- Host on IIS, Azure, DOCKER or Linux.
- Use kestrel as the web servers.
- Enables HTTPS, logging, and diagnostics.

Summary:

- Minimal API - Simple, lightweight services.
- Controllers - Structured and Scalable.
- Advanced features - PATCH, DTOs, Auth, Swagger.
- Deployment - production config and hosting.

Caching in ASP.NET Core:

• What is a cache?

- What a cache does, in the most simple words I can say is: "A cache simply stores the output generated by a page in the memory and this saved output (cache) will serve us (users) in the future." That's it.

• Types of caching:

- (a) Page caching
 - (b) fragment caching
 - (c) Data caching
- } self study

- (a) Page caching: Let's explore the caching of an entire page, first. To cache an entire page's output we need to specify a directive at the top of our page. This directive is the @OutputCache. Let's figure out a simple demo of it.

```
<%@OutputCache Duration = 5    VarByParam = "ID" %>
```

Here, in that statement duration and varByParam are the two attributes of the OutputCache directive. Now let's explore how they work.

- Duration Attribute: This attribute represents the time in seconds of how long the output cache should be stored in memory. After the defined duration the content stored in the memory will be cleared automatically.

- VaryByParam Attribute: This is the most important attribute; you can't afford to miss that in the OutputCache directive Statement. It generally defines the query string parameters to vary the cache (in memory).

You can also multiple parameter names, but for that, you need to separate them using a semi-colon (;). You can also specify it as "*". In this case, the cached content is varied for all the parameters using the query string. for example,

What is JSON?

- JSON (JavaScript Object Notation) is standard design for human-readable data interchange. I think it is 'human-readable' for developers. JSON works with a tree structure and it looks like a XML. It's shorter and very easy to use. If you already have experience with XML, you will certainly learn easily.

JSON vs. XML:

- Example of JSON:

```
{
  "universities": {
    "University": "South Carolina State University",
    "students": [
      { "name": "Stephen Cousins" },
      { "name": "Austin A. Newton" },
      { "name": "Adam Willhite" },
      { "name": "Enis Kurtay YILMAZ" }
    ]
  }
}
```

- Example of XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<universities>
  <university> South Carolina State University
</university>
  <students>
    <name> Stephen Cousins </name>
  </students>
  <students>
    <name> Austin A. Newton </name>
  </students>
  <students>
    <name> Adam Willhite </name>
  </students>
  <students>
    <name> Enis Kurtay YILMAZ </name>
  </students>
</universities>
```

How to create JSON String in C#:

- Please create your new console project from Visual Studio.
- Click File, New, Project, then Console Application (.NET Framework 3.5).
- If you want to create or read a JSON string, we need a JSON Serialize or Deserialize. So, please open your Solution Explorer in Visual Studio, right click on References, and then click "Manage NuGet Package."
- Please search "Newtonsoft.Json" on NuGet Package Manager and install it.
- Please add "using Newtonsoft.Json;" statement. If you forget to add this statement, you can't serialize any

JSON Strings (Example)

- Now, you need to create class for JSON. If you want to create easily, you can use the website jsonutils.com.

How to Get Data from JSON File:

- Make JSON FILE Eg. C:\File\Rishi.json

JSON Code: {

"students": [

{

"Name": "John",

"Surname": "Doe",

"Age": 20,

"Jobs": ["Developer", "Designer"]},

{

"Name": "Jane",

"Surname": "Smith",

"Age": 22,

"Jobs": ["Manager"]

}

}]

- Type this code on Console Application and select the Version 12.0.3

using Newtonsoft.Json;

using System;

using System.Collections.Generic;

using System.IO;

namespace GetDataFromJSONFile {
public class Student {

public string Name {get; set;}

public string Surname {get; set;}

public int Age {get; set;}

public List<string> Jobs {get; set;}

}

public class Root {

public List<Student> Students {get; set;} // changed
// property name to Students

}

class Program {

static void Main (String[] args) {

String path = @"C:\File\Rishi.json";

// corrected variable name to 'path'

// Read the JSON file directly

String jsonContent;

try {

jsonContent = File.ReadAllText(path);

} catch (IOException ex) {

console.WriteLine(\$"Error reading file :
{ex.Message}");

return; // Exit if there's an error reading
the file

}

// Deserialize the JSON context

Root root = JsonConvert.DeserializeObject<Root>
(jsonContent);

// Check if root or Students list is null

if (root?.Students != null) // Adjusted for the
renamed property {

// Iterate through the list of Students

foreach (Student s in root.Students) {

console.WriteLine(\$"{s.Name} {s.Surname}");


```
Age: {s.Age}");  
if (s.Jobs != null) {  
    foreach (var job in s.Jobs) {  
        Console.WriteLine(job);  
    }  
}  
}  
else {  
    Console.WriteLine("No Students found.");  
    Console.ReadKey();  
}  
}
```